

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**WEB APPLICATION DEVELOPMENT
IT093IU
LAB 5**

Member List:

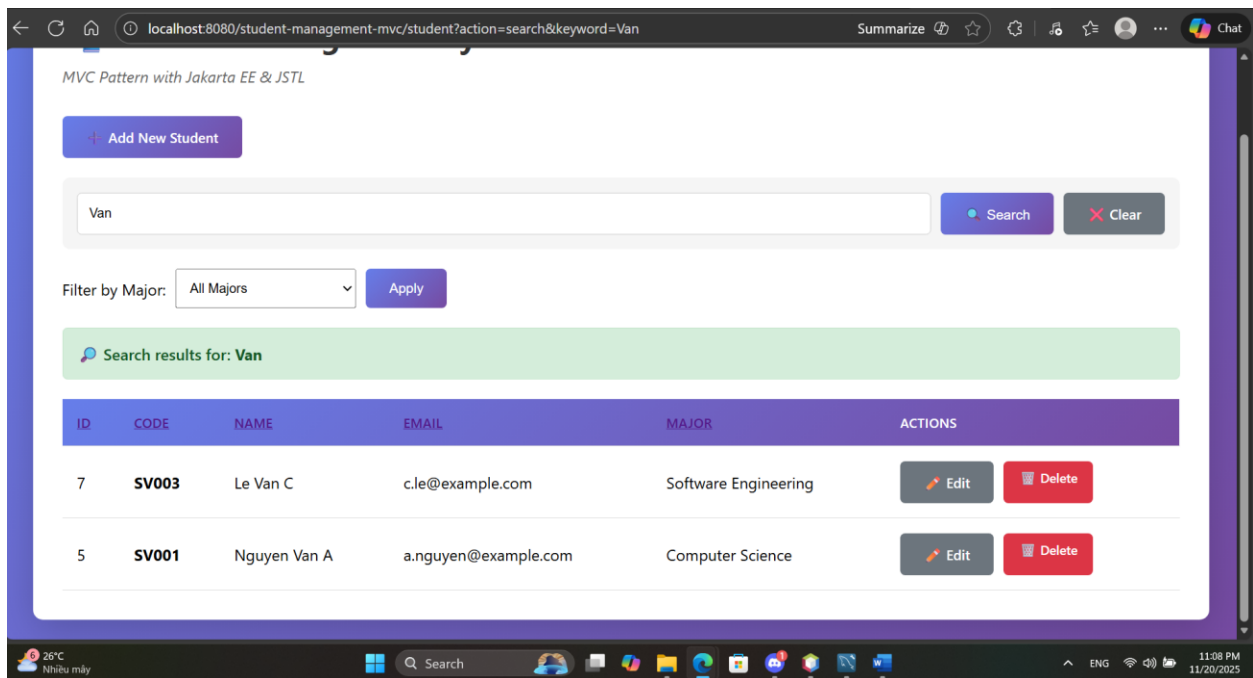
- Huỳnh Ngọc Bảo Long – ITITWE23023

Instructor: MSc. Nguyễn Trung Nghĩa

Exercise 5 – Search functionality

Scenario: User searches for “Van”

- Browser sends request
 - User types “Van” in search box → clicks  Search
 - Browser → GET /student?action=search&keyword=Van
- Controller receives request
 - StudentController.doGet() is called
 - Extracts:
 - action = search
 - keyword = Van
 - Controller switches to: case "search": searchStudents()
- Controller calls DAO
 - Inside searchStudents():
 - Controller → studentDAO.searchStudents("Van")
- DAO executes SQL
 - DAO builds SQL using LIKE operator:
SELECT * FROM students
WHERE student_code LIKE '%Van%'
OR full_name LIKE '%Van%'
OR email LIKE '%Van%'
ORDER BY id DESC;
 - Database returns matching rows.
 - DAO → returns List<Student>
- Controller prepares response
 - Controller sets attributes:
 - request.setAttribute("students", list);
 - request.setAttribute("keyword", "Van");
 - Forwards to: /views/student-list.jsp
- View (JSP) displays results
 - JSP loops using: <c:forEach var="student" items="\${students}">
 - Search box keeps keyword: value="\${keyword}"
- Browser receives final HTML
 - User sees filtered list
 - “Search results for: Van”



Exercise 6 – Server-side validation

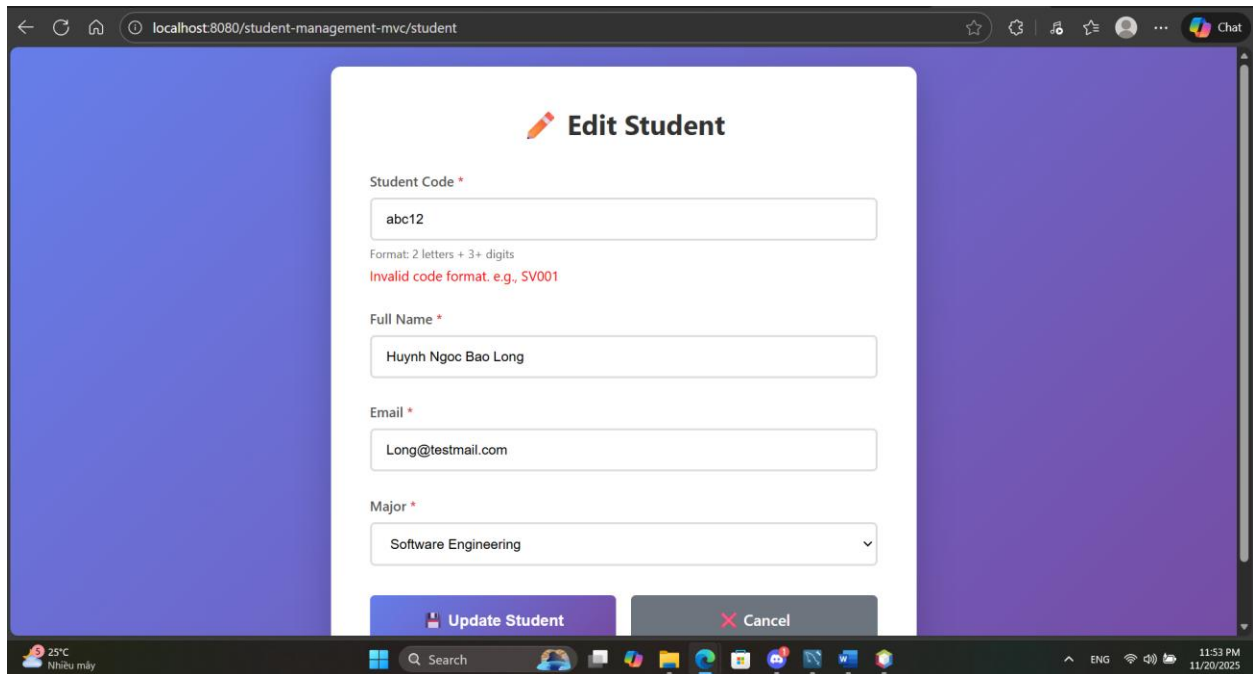
Scenario: User submits a form with invalid Student Code

- Browser submits form
 - User enters “abc12” (invalid code)
 - Clicks Add Student
 - Browser → POST /student?action=insert
 - Form fields included.
- Controller receives request
 - Action = insert
 - Controller extracts form data.
 - Creates new Student object.
- Controller performs validation
 - Controller calls: `validateStudent(student, request)`
 - Validation checks:
 - Student Code matches AA000 format? → fails
 - Full Name not empty? → pass
 - Email valid? → pass
 - Major chosen? → pass
 - Validation returns false.
 - Controller sets error attributes:

- request.setAttribute("errorCode", "Invalid student code format");
 - request.setAttribute("student", student);
- Controller DOES NOT call DAO
 - Because input is invalid, controller:
 - stops processing
 - forwards back to form
 - dispatcher.forward(request, response);
- JSP shows error messages
 - Inside student-form.jsp

```
<c:if test="${not empty errorCode}">
    <span class="error">${errorCode}</span>
</c:if>
```
 - Form is pre-filled with previously entered data:
value="\${student.studentCode}"
- Browser receives the error form
 - User sees: Invalid student code format
 - And the form stays filled.

The screenshot shows a web browser window at the URL `localhost:8080/student-management-mvc/student?action=new`. The page has a purple header and a white form titled "+ Add New Student". The form contains four input fields: "Student Code" (with value "abc12" and a red asterisk), "Full Name" (with value "Huynh Ngoc Bao Long" and a red asterisk), "Email" (with value "Long@testmail.com" and a red asterisk), and "Major" (with value "Software Engineering" and a red asterisk). Below the fields are two buttons: "+ Add Student" (purple) and "X Cancel" (grey). The browser's address bar shows the URL, and the Windows taskbar at the bottom shows the time as 11:15 PM on 11/20/2023.



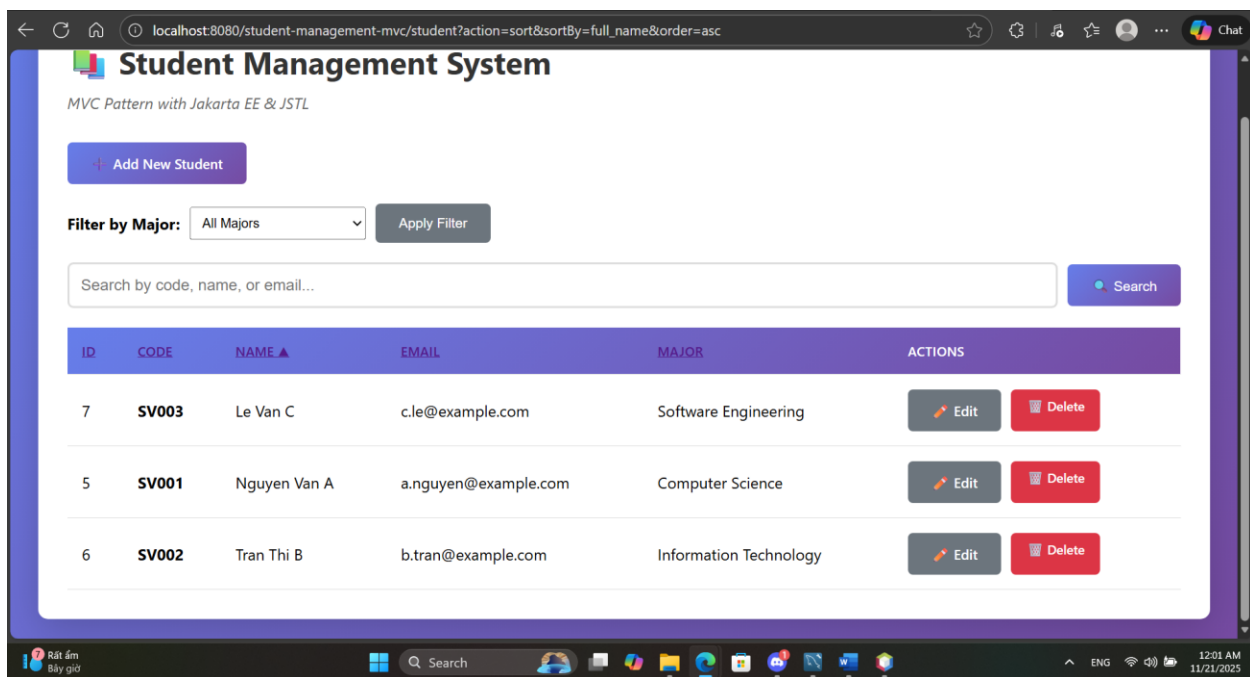
Exercise 7 – Sorting & filtering

Scenario A: User clicks “Name” column to sort ascending

- Browser sends request
 - User clicks: NAME
 - Browser → GET /student?action=sort&sortBy=full_name&order=asc
- Controller receives request
 - Controller extracts:
 - action = sort
 - sortBy = full_name
 - order = asc
 - Calls: sortStudents(request, response)
- Controller calls DAO
 - studentDAO.getStudentsSorted("full_name", "asc")
 - DAO validates parameters
 - DAO builds SQL:

```
SELECT * FROM students
ORDER BY full_name ASC;
```
 - DAO returns sorted List<Student>.
- Controller forwards to JSP
 - Sets:

- request.setAttribute("students", list);
- request.setAttribute("sortBy", "full_name");
- request.setAttribute("order", "asc");
- Forwards to: student-list.jsp
- JSP displays sorted table
 - Header links use the exercise's format: `<a href="student?action=sort&sortBy=full_name&order=asc">NAME</a>`
 - Rows appear alphabetically sorted.
- Browser receives sorted HTML
 - User sees sorted student list.



Scenario B: User filters by Major: “Software Engineering”

- Browser sends request
 - User selects major → clicks Apply
 - Browser → GET /student?action=filter&major=Software+Engineering
- Controller handles filter
 - Extracts:
 - action = filter
 - major = "Software Engineering"
 - Calls: filterStudents()
- Controller → DAO

- `studentDAO.getStudentsByMajor("Software Engineering")`
- DAO runs:

```
SELECT * FROM students
WHERE major = 'Software Engineering'
ORDER BY id DESC;
```
- DAO returns filtered list.
- Controller forwards to JSP
 - Sets:

```
selectedMajor = "Software Engineering"
students = filteredList
```
- JSP updates dropdown
 - Dropdown keeps selected major:

```
<option value="Software Engineering"
    ${selectedMajor == 'Software Engineering' ? 'selected' : ''}>
```
- Browser displays filtered results
 - Only Software Engineering students appear.

